

PRAKTIKUM SISTEM BASIS DATA

Nama	Jonathan Frederick Kosasih	No. Modul	6
NPM	2306225981	Tipe	TP

1. Hashing adalah sebuah proses untuk menghasilkan sebuah output dengan ukuran fix dari sebuah input dengan ukuran variatif menggunakan formula matematis yang dikenal sebagai fungsi fungsi hash. Hashing umum digunakan pada programming untuk mengelola penyimpanan data agar mudah diakses dan disimpan, juga umum digunakan pada kriptografi untuk menjaga integritas data sensitif seperti kata sandi atau data pribadi lainnya.

Beberapa jenis algoritma hashing yang umum digunakan:

- Secure Hash Algorithm (SHA) membuat sebuah nilai hash unik dari input secara satu arah yang biasa digunakan untuk mengidentifikasi perubahan atau kerusakan pada teks.
- Message Digest 5 (MD5) menghasilkan sebuah nilai berukuran 128-bit dari input yang ukurannya variatif.
- Bcrypt dirancang secara khusus untuk proses hashing pada p=kata sandi yang penyimpanannya pada aplikasi backend secara aman.

Berikut kelebihan dan kekurangan masing-masing algoritma:

SHA	MD5	bcrypt
Kelebihan: - Lebih resisten terhadap serangan hash collision - Tidak dapat disangkal	Kelebihan: - Cepat dan mudah untuk diimplementasikan - Penggunaan memori rendah	Kelebihan: - Built-in salting → output sudah terjamin unik - Proses hashing lambat → kebal terhadap brute force - Adaptif terhadap hardware
Kekurangan: - Hashing terlalu cepat → mudah dibobol dengan algoritma brute force - Tidak memiliki salting	Kekurangan: - Rentan terhadap serangan hash collision - Outdated	Kekurangan: - Proses cukup lambat → tidak cocok apabila hashing sering dilakukan - Konsumsi memori tinggi

Referensi:

- Kamal, “Advanced Express”, *Netlab DTE*, [Online]. Available: <https://learn.netlabdte.com/docs/SBD/Advanced%20Express>. [Accessed: 18 March 2025].
- Author, “What is Hashing?”, *GeeksforGeeks*, [Online]. Available: <https://www.geeksforgeeks.org/what-is-hashing/>. [Accessed: 18 March 2025].
- Author, “How Does a Secure Hash Algorithm work in Cryptography?”, *GeeksforGeeks*, [Online]. Available: <https://www.geeksforgeeks.org/how-does-a-secure-hash-algorithm-work-in-cryptography/>. [Accessed: 18 March 2025].
- Author, “What is the MD5 Algorithm?”, *GeeksforGeeks*, [Online]. Available: <https://www.geeksforgeeks.org/what-is-the-md5-algorithm/>. [Accessed: 18 March 2025].
- Monika Grigutyte, “What is bcrypt and how does it work?”, *NordVPN*, [Online]. Available: <https://nordvpn.com/blog/what-is-bcrypt/>. [Accessed: 18 March 2025].
- Author, “Types Of Hashes”, *FasterCapital*, [Online]. Available: <https://fastercapital.com/topics/types-of-hashes.html>. [Accessed: 18 March 2025].
- Athreya aka Maneshwar, “Hashing Passwords: Why MD5 and SHA Are Outdated, and Why You Should Use Scrypt or Bcrypt”, *Dev.to*, [Online]. Available: <https://dev.to/lovestaco/hashing-passwords-why-md5-and-sha-are-outdated-and-why-you-should-use-scrypt-or-bcrypt-48p2>. [Accessed: 18 March 2025].

2. Middleware adalah sebuah software yang berada di antara aplikasi dengan server sebagai perantara komunikasi antara kedua pihak tersebut. Middleware menangkap request dan response, serta melakukan beberapa operasi sebelum meneruskannya ke router handler terakhir. Middleware menangani berbagai tugas seperti data translation, message queuing, authentication, dan connectivity; sehingga memudahkan proses integrasi dan pengelolaan software environment.

Contoh middleware antara lain:

- **Database middleware:** membuat aplikasi dapat mengakses dan berinteraksi dengan database. Contoh: Oracle Fusion Middleware
- **Application Server Middleware:** menyediakan platform untuk menjalankan dan mengelola aplikasi dan layanan web. Contoh: IBM WebSphere Application Server

- **Message-Oriented Middleware:** memfasilitasi komunikasi asinkron antar aplikasi menggunakan message queue dan brokers. Contoh: Microsoft BizTalk Server
- **Web Middleware:** menangani request dan response web, termasuk tugas seperti routing, caching, dan security. Contoh: Apache Tomcat servers
- **API Middleware:** menyediakan peralatan untuk membuat, mengekspos, dan mengelola API agar aplikasi berbeda dapat berinteraksi satu dengan lainnya.

Referensi:

- Author, “Middleware: Categories, Types, working, Advantage and Disadvantage”, *GeeksforGeeks*, [Online]. Available: <https://www.geeksforgeeks.org/what-is-middleware/>. [Accessed: 18 March 2025].
- Vivek Chavda, “Express.js Middleware: A Secret Ingredient to Build Scalable Web Applications”, *Radix*, [Online]. Available: <https://radixweb.com/blog/expressjs-middleware>. [Accessed: 18 March 2025].
- Author, “What Is Middleware? Definition, Guide & Examples”, *Okta*, [Online]. Available: <https://www.okta.com/identity-101/middleware/>. [Accessed: 18 March 2025].

3. Bagian dari source code untuk mengkonfigurasi middleware CORS sesuai dengan ketentuan soal:

```
const express = require("express");
const cors = require("cors");
const app = express();

const corsOptions = {
  origin: "https://os.netlabdte.com",
  methods: ["GET", "POST", "PUT", "DELETE"],
};

app.use(cors(corsOptions));
```

Referensi:

- Kamal, “Advanced Express”, *Netlab DTE*, [Online]. Available: <https://learn.netlabdte.com/docs/SBD/Advanced%20Express>. [Accessed: 18 March 2025].
- Author, “cors”, *NPM*, [Online]. Available: <https://www.npmjs.com/package/cors>. [Accessed: 18 March 2025].

4. Regular Expressions (Regex) adalah sebuah urutan karakter yang mendefinisikan sebuah pola searching. Regex seringkali digunakan untuk mencocokkan, mencari, atau memanipulasi string, sehingga sangat berguna untuk validasi input, pencarian teks, atau penggantian teks dalam berbagai aplikasi. Regex diimplementasikan pada berbagai bahasa pemrograman dan text editor, sehingga menjadi sebuah alat dengan banyak pengaplikasiannya.

Referensi:

- Kamal, “Advanced Express”, *Netlab DTE*, [Online]. Available: <https://learn.netlabdte.com/docs/SBD/Advanced%20Express>. [Accessed: 18 March 2025].
- Author, “Regex Tutorial – How to write Regular Expressions?”, *GeeksforGeeks*, [Online]. Available: <https://www.geeksforgeeks.org/write-regular-expressions/>. [Accessed: 18 March 2025].
- Joy Victor, “Understanding Regular Expressions (Regex)”, *Medium*, [Online]. Available: <https://medium.com/@victoriousjvictor/understanding-regular-expressions-regex-e1c048f5aa6c>. [Accessed: 18 March 2025].

5. Contoh regex untuk memeriksa email:

```
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

Referensi:

- Kamal, “Advanced Express”, *Netlab DTE*, [Online]. Available: <https://learn.netlabdte.com/docs/SBD/Advanced%20Express>. [Accessed: 18 March 2025].
- Author, “JavaScript RegExp Reference”, *W3Schools*, [Online]. Available: https://www.w3schools.com/jsref/jsref_obj_regexp.asp. [Accessed: 18 March 2025].

6. Contoh regex untuk memeriksa password sesuai dengan ketentuan soal:

```
const passRegex = /^(?=.*[^\s]{8,})(?=.*\d)(?=.*[^\w\d]).+$/;
```

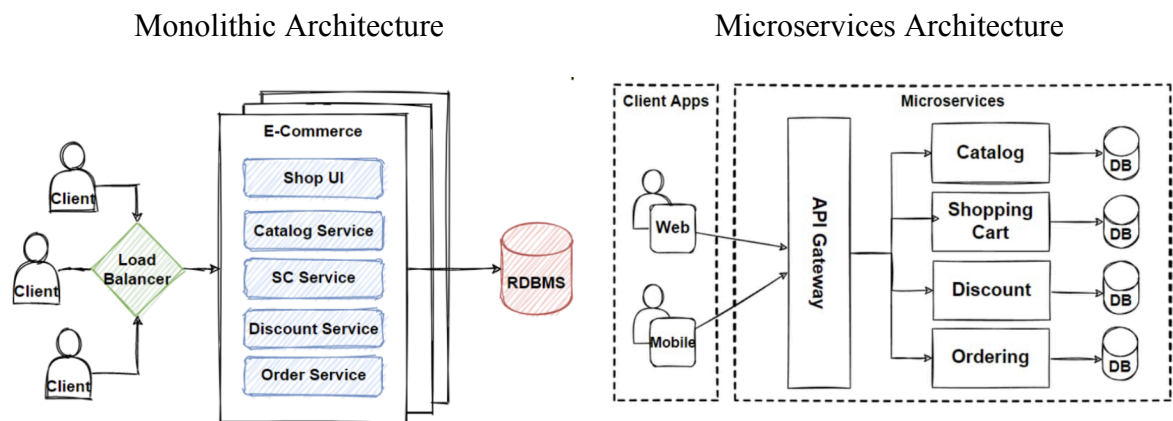
Referensi:

- Kamal, “Advanced Express”, *Netlab DTE*, [Online]. Available: <https://learn.netlabdte.com/docs/SBD/Advanced%20Express>. [Accessed: 18 March 2025].
- Author, “JavaScript RegExp Reference”, *W3Schools*, [Online]. Available: https://www.w3schools.com/jsref/jsref_obj_regexp.asp. [Accessed: 18 March 2025].

7. Sebuah backend dapat dibuat scalable dan secure melalui beberapa hal, di antaranya:

- **Membangun fondasi tepat dengan arsitektur backend yang sesuai.**

Terdapat 2 arsitektur yang umum digunakan, yaitu Monolithic Architecture dan Microservices Architecture.



- **Membuat aplikasi kita stateless agar lebih scalable secara horizontal.**

Statelessness berarti setiap server tidak perlu mengingat sesi user, sehingga memudahkan kita untuk menambahkan atau menghilangkan server tanpa mengganggu aplikasi yang sedang berjalan karena setiap server tidak bergantung satu dengan lainnya. REST API bersifat stateless secara desainnya, sehingga memudahkan scaling.

- **Menambahkan smart traffic distribution.**

Smart traffic distribution dapat dilakukan dengan implementasi load balancing untuk mendistribusikan request yang datang secara merata ke beberapa server untuk memastikan tidak ada server yang kewalahan dan mencegah terjadinya slowdown atau crash.

- **Mengoptimasi database agar tidak menjadi bottleneck.**

Untuk mengoptimalkan database, kita dapat melakukan Indexing, Caching, Database Sharding, Database Partitioning, dan Connection Pooling

- **Mengimplementasikan smart monitoring dan logging untuk mencegah backend meltdown.**

Smart monitoring dan logging dapat membantu menjaga kesehatan dari backend dengan menangkap masalah sebelum lepas dari kendali. Tanpa adanya monitoring yang baik, backend kita bisa kewalahan karena adanya traffic yang melebihi kapasitas secara mendadak.

- **Mengotomatisasikan penyediaan infrastruktur dan DevOps.**

Setup server secara manual, konfigurasi environment, atau deploy kode terkadang dapat memperlambat aplikasi sambil mengembangkan skala aplikasi tersebut. Proses otomatisasi mengeliminasi sebagian besar dari tugas yang repetitif dan meningkatkan konsistensi.

- **Mengimplementasikan caching.**

Caching dilakukan untuk menyimpan data yang sering diakses secara sementara agar request mendatang dapat dilayani dengan lebih cepat tanpa mengulang query pada database atau memproses ulang informasi tersebut.

Referensi:

- Kamal, “Advanced Express”, *Netlab DTE*, [Online]. Available: <https://learn.netlabdte.com/docs/SBD/Advanced%20Express>. [Accessed: 18 March 2025].
- Anmol Baranwal, “7 steps to building scalable Backend from scratch”, *Dev.to*, [Online]. Available: <https://dev.to/anmolbaranwal/7-steps-to-building-scalable-backend-from-scratch-fp8>. [Accessed: 18 March 2025].